
TRANSPORT IT

Transport Industry Projects

A Portfolio of IT Solutions for Data Analytics & Software Development

Project Proposal

Transportation Security System

Prepared By: Ashley Chindanga
Date: August 3, 2026
Version: 2.0
Status: Final
Project Type: Software Development
Estimated Time: 4-6 Weeks

Project Code: TSS-2026-01
Confidential - For Review Only

This document is confidential and intended solely for the use of the individual or entity to whom it is addressed.

Contents

1	Introduction	2
1.1	Background	2
1.2	Project Overview	2
1.3	Project Objectives	2
1.4	Target Audience	2
2	Problem Statement	3
2.1	Current Challenges	3
2.2	Why This Project Matters	3
3	Technical Scope	3
3.1	System Architecture	3
3.2	Data Flow	4
3.3	Core Modules	5
3.3.1	1. Data Collection Module	5
3.3.2	2. Driver Monitoring Module (Computer Vision)	5
3.3.3	3. Behavior Analysis Module	6
3.3.4	4. Alert and Notification Module	6
3.3.5	5. Dashboard Module	6
3.4	Technologies Used	6
3.5	Data Sources	7
4	Implementation Plan	7
4.1	Development Phases	7
4.2	Detailed Milestones	8
5	Deliverables	8
5.1	Code Repository	8
5.2	Documentation	8
5.3	Working Software	8
5.4	Portfolio Assets	9
6	Skills Demonstrated	9
7	Risk Assessment	10
8	Success Metrics	10
9	Conclusion	10
A	Appendix A: Sample Code Structure	12
B	Appendix B: Installation Guide	13
C	Appendix C: Requirements File	13
D	Appendix D: References	14

1 Introduction

1.1 Background

The transportation and logistics industry is experiencing rapid digital transformation. With increasing concerns about driver safety, regulatory compliance, and operational efficiency, there is a growing demand for intelligent security systems that can monitor driver behavior and ensure safety on the roads.

According to the World Health Organization, approximately 1.3 million people die each year as a result of road traffic crashes, with driver fatigue and distraction being leading causes. This project addresses these critical safety challenges through technology.

1.2 Project Overview

The **Transportation Security System** is a comprehensive IT solution designed to monitor driver behavior, detect unsafe driving patterns, and prevent accidents before they occur. The system leverages:

- **Computer Vision** for driver fatigue and distraction detection
- **IoT Sensors** for vehicle telemetry data collection
- **Real-time Analytics** for immediate threat detection
- **Alert Systems** for proactive safety interventions

1.3 Project Objectives

1. Develop a real-time driver monitoring system using computer vision
2. Implement algorithms to detect unsafe driving behaviors
3. Create an automated alert system for safety violations
4. Build a comprehensive dashboard for safety analytics
5. Generate driver scorecards and safety reports

1.4 Target Audience

- **Fleet Managers** - Monitor driver performance across fleets
- **Transport Companies** - Ensure regulatory compliance
- **Safety Officers** - Track and improve safety metrics
- **Drivers** - Receive real-time feedback on driving behavior
- **Insurance Companies** - Access driving data for risk assessment

2 Problem Statement

2.1 Current Challenges

The transportation industry faces significant safety challenges:

Challenge	Impact	Industry Data
Driver Fatigue	20-30% of all road accidents	WHO, 2023
Distracted Driving	25% of all traffic accidents	NHTSA, 2023
Speeding	30% of fatal accidents	IIHS, 2023
Harsh Braking	Increased vehicle wear, passenger discomfort	Industry Report
Lack of Monitoring	Delayed incident response, no preventive action	Industry Report

Table 1: Key Safety Challenges in Transportation

2.2 Why This Project Matters

Benefit	Category	Description
Safety First	Critical	Prevent accidents and save lives through early intervention
Cost Reduction	Financial	Reduce maintenance costs from harsh driving behavior
Compliance	Regulatory	Meet regulatory safety requirements and avoid penalties
Insurance Benefits	Financial	Lower premiums through documented safety improvements
Driver Improvement	Training	Provide actionable feedback for driver development

Table 2: Key Benefits of the Transportation Security System

3 Technical Scope

3.1 System Architecture

The Transportation Security System follows a modular architecture with five core components arranged in a data pipeline:

Module	Technology	Function	Output
Data Collection	GPS, Camera, IoT Sensors	Captures real-time vehicle/driver data	Raw data stream
Processing & Analysis	OpenCV, Python, Dlib	Analyzes data for safety violations	Detected events
Alert System	WebSockets, SMTP	Sends multi-channel notifications	Alerts
Database	PostgreSQL/SQLite	Stores all system data	Historical data
Dashboard & Visualization	Flask/Dash, Plotly	Visualizes metrics & alerts	User interface

Table 3: System Architecture Components

3.2 Data Flow

The system follows a clear, unidirectional data flow:

1. Step 1: Data Collection

- GPS sensors provide real-time location and speed data
- Accelerometer detects harsh braking and cornering events
- Camera captures driver's face for fatigue detection

2. Step 2: Processing & Analysis

- Computer vision algorithms detect fatigue using Eye Aspect Ratio (EAR)
- Behavior analysis detects speeding, harsh braking, and distracted driving
- All analysis occurs in real-time with less than 500ms latency

3. Step 3: Alert Generation

- When a violation is detected, an alert is triggered immediately
- Alerts are sent via Email, SMS, and Push notifications
- Severity levels: CRITICAL, WARNING, INFO

4. Step 4: Data Storage

- All raw and processed data is stored in PostgreSQL
- Real-time data is available via WebSockets
- Historical data supports reporting and analytics

5. Step 5: Visualization

- Real-time dashboard enables monitoring
- Historical reports and trend analysis
- Driver scorecards and safety leaderboards

Step	Process	Key Technologies	Latency
1	Data Collection	GPS, Camera, MQTT	< 100ms
2	Processing & Analysis	OpenCV, Dlib, Python	< 500ms
3	Alert Generation	WebSockets, SMTP, Twilio	< 200ms
4	Data Storage	PostgreSQL, SQLite	< 100ms
5	Visualization	Flask, Plotly, Dash	< 300ms

Table 4: Data Flow - Processing Times

3.3 Core Modules

3.3.1 1. Data Collection Module

Data Source	Data Collected	Purpose
GPS Module	Speed, Location, Route	Speeding detection, route tracking
Accelerometer	Acceleration, Braking, Cornering	Harsh braking, cornering detection
Camera (Webcam)	Driver face video	Fatigue detection, distraction detection
Vehicle OBD-II	Fuel, Engine data (optional)	Vehicle health monitoring

Table 5: Data Collection Module Specifications

3.3.2 2. Driver Monitoring Module (Computer Vision)

Feature	Algorithm/Technology	Detection Criteria
Face Detection	Dlib / Haar Cascade	Locate face in frame
Facial Landmarks	Dlib 68-point predictor	Identify eyes, mouth, head pose
Eye Blink Detection	Eye Aspect Ratio (EAR)	$EAR < 0.25$ for 3+ frames
Yawning Detection	Mouth Aspect Ratio (MAR)	$MAR > 0.6$ for 2+ frames
Head Pose Estimation	Rotation vectors	Head tilted $> 30^\circ$
Distraction Detection	Object detection	Phone usage, looking away

Table 6: Driver Monitoring - Computer Vision Techniques

3.3.3 3. Behavior Analysis Module

Behavior	Measurement	Threshold
Speeding	GPS Speed vs Road Limit	Speed > Limit + 10%
Harsh Braking	Deceleration Rate	> 3 m/s ²
Harsh Cornering	Lateral Acceleration	> 2 m/s ²
Fatigue	Eye Aspect Ratio (EAR)	EAR < 0.25 for 3+ seconds
Distraction	Head Position	Looking away > 2 seconds
Aggressive Driving	Combined Metrics	Composite score < 40

Table 7: Behavior Detection Thresholds

3.3.4 4. Alert and Notification Module

Severity	Trigger	Notification	Action
CRITICAL	Fatigue Detected	SMS + Email + Push	Immediate driver alert
WARNING	Speeding, Harsh Braking	Email + Push	Manager notification
INFO	Route deviation	Push only	Logged for review

Table 8: Alert Severity and Response

3.3.5 5. Dashboard Module

The dashboard provides real-time visibility with the following sections:

Section	Metrics Displayed	Update Frequency
KPI Cards	Safety Score, Active Alerts, Driver Status	Real-time (1s)
Behavior Trends	Bar charts for violations over time	5 minutes
Recent Incidents	List of latest alerts with severity	Real-time
Driver Ranking	Leaderboard by safety score	1 hour
Reports	Historical analytics and reports	On-demand

Table 9: Dashboard Components

3.4 Technologies Used

Category	Technology	Purpose
Programming	Python 3.9+	Core development
Computer Vision	OpenCV, Dlib, Face Recognition	Face detection, landmark tracking

Machine Learning	TensorFlow / Py-Torch	Fatigue detection model training
Web Framework	Flask / Dash	Dashboard application
Database	PostgreSQL / SQLite	Data storage
Real-time Communi-cation	WebSockets, MQTT	Live data streaming
Visualization	Plotly, Matplotlib, Seaborn	Charts, graphs, dashboards
Testing	Pytest, Unittest, Mock	Unit testing, integration testing
Version Control	Git, GitHub	Source code management
CI/CD (Optional)	GitHub Actions	Automated testing
Containerization	Docker (Optional)	Deployment consistency

Table 10: Technology Stack

3.5 Data Sources

Data Source	Type	Purpose
Synthetic Data	Generated using simula-tors	Initial development phase
UCI Driving Dataset	Public dataset	Model training & valida-tion
NTHSA Crash Data	Public dataset	Statistical analysis
Custom GPS Simu-lator	Python simulation	GPS data simulation
Webcam Feed	Real-time video	Driver monitoring testing

Table 11: Data Sources

4 Implementation Plan

4.1 Development Phases

Phase	Activities	Duration	Deliverables
Phase 1	Project setup, data collec-tion, sensor simulation	Week 1	Working data pipeline, project structure
Phase 2	Computer vision imple-mentation (face detection, fatigue)	Week 2-3	Fatigue detection module working
Phase 3	Behavior analysis algo-rithms (speeding, braking)	Week 3-4	All detectors func-tional
Phase 4	Alert system and dash-board development	Week 4-5	Full interface com-plete
Phase 5	Testing, integration, docu-mentation	Week 5-6	Complete system ready

Table 12: Development Timeline

4.2 Detailed Milestones

Week	Milestone	Success Criteria
Week 1	Project Structure & Data Pipeline	All folders created, data flowing
Week 2	Face Detection & Tracking	Face detected in 95% of frames
Week 3	Fatigue Detection Functional	EAR detection working, >85% accuracy
Week 4	Behavior Analysis Complete	All 5 detectors functional
Week 5	Dashboard & Alerts Complete	UI displays real-time data
Week 6	Testing & Documentation Complete	All tests passing, docs complete

Table 13: Project Milestones

5 Deliverables

5.1 Code Repository

- Complete source code on GitHub with version control
- Well-structured project with clear module separation
- Modular code following PEP8 standards
- Comprehensive README with setup and installation instructions

5.2 Documentation

Document	Description
Project Proposal	This document - project overview and justification
Technical Requirements	Detailed functional and non-functional requirements
System Architecture	Technical design and component interactions
API Documentation	Endpoints, request/response formats
User Guide	Installation, setup, and usage instructions
Developer Guide	Code structure and contribution guidelines

Table 14: Documentation Deliverables

5.3 Working Software

- Real-time driver monitoring system
- Web-based dashboard for safety analytics
- Automated multi-channel alert system
- Driver scorecard and reporting system

5.4 Portfolio Assets

- Demo video (5 minutes) showing system in action
- Screenshots of working dashboard and features
- Presentation slides for job interviews
- GitHub repository with detailed README

6 Skills Demonstrated

Skill Category	Specific Skills	Project Application
Programming	Python, OOP, API Development	All backend code
Computer Vision	OpenCV, Dlib, Image Processing	Fatigue detection
Data Analytics	Real-time Processing, Statistics	Behavior analysis
IoT	Sensor Data, MQTT Protocol	Data collection
Web Development	Flask/Dash, HTML/CSS, JS	Dashboard
Database	SQL, Data Modeling	Data storage
Machine Learning	Classification, Model Training	Fatigue model
Visualization	Plotly, Matplotlib, Dashboards	Analytics UI
Security	Data Privacy, Encryption	Driver data protection
Project Management	Documentation, Planning	All phases
Problem Solving	Debugging, Optimization	Throughout project

Table 15: Skills Demonstrated by This Project

7 Risk Assessment

Risk	Impact Level	Mitigation Strategy
Hardware access limitation	Medium	Use simulators for development phase
Model accuracy issues	High	Tune thresholds, use ensemble methods
Performance issues	Medium	Optimize algorithms, implement caching
Integration challenges	Medium	Regular integration testing
Data privacy concerns	High	Anonymize data, local processing only
Dependency conflicts	Low	Use virtual environment, freeze versions
Time constraints	Medium	Prioritize core features, agile approach

Table 16: Risk Assessment and Mitigation

8 Success Metrics

Metric	Target	Measurement Method
Fatigue Detection Accuracy	>85%	Test dataset validation
Processing Speed	<500ms per frame	Performance benchmarking
Alert False Positive Rate	<5%	Manual verification
Dashboard Usability	Intuitive UI	User testing feedback
Code Quality	PEP8 compliant	Flake8/Pylint checks
Test Coverage	>70%	Pytest coverage report
Documentation	Complete	Peer review

Table 17: Success Metrics and Targets

9 Conclusion

The Transportation Security System is a comprehensive IT solution that addresses critical safety challenges in the transportation industry. By leveraging computer vision, IoT sensors, and real-time analytics, this system demonstrates the ability to:

1. Build intelligent monitoring systems for safety
2. Apply computer vision to solve real-world problems
3. Develop real-time analytics applications
4. Create user-friendly safety dashboards
5. Demonstrate industry-relevant technical skills

This project not only creates a valuable safety solution but also builds a strong portfolio demonstrating expertise in computer vision, IoT, data analytics, and software development - exactly what employers in the transportation technology sector are looking for.

Next Steps

1. Set up project structure
2. Install required dependencies
3. Begin Phase 1: Data Collection Module

A Appendix A: Sample Code Structure

Listing 1: Fatigue Detection Class

```

1 import cv2
2 import dlib
3 import numpy as np
4 from scipy.spatial import distance
5
6 class FatigueDetector:
7     """
8     Detects driver fatigue using Eye Aspect Ratio (EAR)
9     """
10    def __init__(self, ear_threshold=0.25, consecutive_frames=3):
11        self.ear_threshold = ear_threshold
12        self.consecutive_frames = consecutive_frames
13        self.frame_counter = 0
14
15        # Initialize face detector and predictor
16        self.detector = dlib.get_frontal_face_detector()
17        self.predictor = dlib.shape_predictor(
18            'shape_predictor_68_face_landmarks.dat'
19        )
20
21    def eye_aspect_ratio(self, eye_landmarks):
22        """Calculate EAR from eye landmarks"""
23        A = distance.euclidean(eye_landmarks[1], eye_landmarks[5])
24        B = distance.euclidean(eye_landmarks[2], eye_landmarks[4])
25        C = distance.euclidean(eye_landmarks[0], eye_landmarks[3])
26        ear = (A + B) / (2.0 * C)
27        return ear
28
29    def detect_fatigue(self, frame):
30        """Main fatigue detection method"""
31        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
32        faces = self.detector(gray)
33
34        for face in faces:
35            landmarks = self.predictor(gray, face)
36
37            # Extract eye landmarks (indices 36-41 for left, 42-47 for right)
38            left_eye = landmarks.parts()[36:42]
39            right_eye = landmarks.parts()[42:48]
40
41            # Calculate EAR for both eyes
42            ear_left = self.eye_aspect_ratio(left_eye)
43            ear_right = self.eye_aspect_ratio(right_eye)
44            ear_avg = (ear_left + ear_right) / 2.0
45
46            # Check if EAR is below threshold
47            if ear_avg < self.ear_threshold:
48                self.frame_counter += 1
49                if self.frame_counter >= self.consecutive_frames:
50                    return True, ear_avg, 'FATIGUE_DETECTED'
51            else:
52                self.frame_counter = 0
53

```

```
54 |         return False, 0, 'NORMAL'
```

B Appendix B: Installation Guide

Listing 2: Setup Commands

```
1 # Clone the repository
2 git clone https://github.com/username/transportation-security-system.
  git
3 cd transportation-security-system
4
5 # Create virtual environment
6 python -m venv venv
7 source venv/bin/activate # Linux/Mac
8 # venv\Scripts\activate # Windows
9
10 # Install dependencies
11 pip install -r requirements.txt
12
13 # Download Dlib face landmark model
14 wget http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2
15 bunzip2 shape_predictor_68_face_landmarks.dat.bz2
16
17 # Set up database
18 python scripts/setup_db.py
19
20 # Run the application
21 python src/dashboard/app.py
```

C Appendix C: Requirements File

Listing 3: requirements.txt

```
1 # Core
2 python>=3.9
3 opencv-python>=4.8.0
4 dlib>=19.24.0
5 numpy>=1.24.0
6 scipy>=1.10.0
7
8 # Machine Learning
9 tensorflow>=2.12.0
10 scikit-learn>=1.2.0
11
12 # Web Framework
13 flask>=2.2.0
14 dash>=2.9.0
15 plotly>=5.13.0
16
17 # Database
18 psycopg2-binary>=2.9.0
19 sqlalchemy>=2.0.0
20
21 # Real-time
22 websockets>=11.0.0
```

```
23 paho-mqtt>=1.6.0
24
25 # Testing
26 pytest>=7.2.0
27 pytest-cov>=4.0.0
28
29 # Visualization
30 matplotlib>=3.6.0
31 seaborn>=0.12.0
```

D Appendix D: References

1. OpenCV Documentation: <https://docs.opencv.org/>
2. Dlib Documentation: <http://dlib.net/>
3. WHO Road Safety Report: <https://www.who.int/roadsafety>
4. NHTSA Crash Data: <https://www.nhtsa.gov/>
5. Flask Documentation: <https://flask.palletsprojects.com/>
6. Dash Documentation: <https://dash.plotly.com/>
7. TensorFlow Documentation: <https://www.tensorflow.org/>